# IE 332 Assignment 2

Group of 3–4 • 15 points

**Due: Monday, October 19, 11:59 p.m. (Gradescope)**

## Instructions

- **Group work (3–4).** Work with your group members and submit *one* notebook per group. Everyone is responsible for understanding every part.
- **One grade per group.** Every member receives the group's grade. If your group runs into a contribution problem, contact the head TA.
- **Contribution table (required, page 1 of your PDF).** The last page of this handout is a contribution table. Fill it in for your group and make it the **first page of the PDF you submit**, ahead of your notebook. Do not put it in the notebook.
- **AI tools are allowed** (ChatGPT, Claude, Copilot, . . . ), with the **AI Reflection** below. Use AI to learn, not to replace your thinking.
- **AI Reflection (required, first cell of your notebook; 0.25 pt).** A few sentences: (1) *how* your group used AI and where it helped; (2) what you *learned* from using it; (3) anything it *got wrong* that you caught and fixed, or state that your group did not identify an error. One line if you used none. It is checked for completion, not marked for content. **Using AI and omitting it here is academic misconduct.**
- **Points.** Part A is 3.5, Part B is 7.5, and Part C is 3.5. The final 0.5 is split equally: **AI Reflection (0.25)** and **reproducibility (0.25)**. Reproducible means that all code cells run top to bottom, in order, using only an unmodified copy of `nba_shots.csv` and the specified settings.
- **Submission: two files to Gradescope, one submission per group.** (1) one **combined PDF**: the filled contribution table as page 1, followed by the printed notebook; this PDF is the source for all answer grading; (2) the Colab notebook itself, `.ipynb`, which we use only to re-run all code for the reproducibility point.
- **Except for the contribution table, everything you want graded goes in the notebook**: your Python code and visible outputs in code cells, every written answer in a Markdown cell next to the question it answers, and your labeled plots. Do not submit a separate write-up.
- **Making the PDF (do this last, and check it).** Run every cell so the outputs are visible, then in Colab use `File > Print > Save as PDF`. Put the filled-in contribution table in front of it, so your PDF reads: **contribution table, then notebook**. You can build the table in a Word or Google doc, export it to PDF, and insert it as page 1. Open the PDF before you submit it and confirm that everything in your notebook is actually in the PDF. **All answer content is graded from the PDF; if the TAs cannot see an answer there, it is graded as missing.** The `.ipynb` is used only for the reproducibility check. Then tag your pages to the questions in Gradescope.
- **After any applicable slip day, late work is not accepted** (see the syllabus policy).

**Coding and display.** Use Python/Pandas and scikit-learn; no SQL is needed. You may reuse lecture or Lab 6 code with a brief source note. Label sections Q1–Q6. Display rates, probabilities, accuracies, and percentage-point differences to 3 decimals unless stated otherwise. Keep unrounded values for calculations and model selection.

## Part A: Understand and Prepare the Data (3.5 points)

Predict whether a shot will be made using information available **before the shot**. Use the unmodified `nba_shots.csv`, with one shot per row.

| Column | Meaning |
|---|---|
| `made` | Target recorded after the shot (not an input): 1 = made, 0 = missed. |
| `dist_ft` | Distance from the shooter to the basket, in feet. |
| `defender_ft` | Distance from the shooter to the nearest opposing player, in feet. |
| `shot_clock` | Seconds left on the team's countdown to attempt a shot (0.3–24 here). This is not the time left in the quarter. |
| `period` | Quarter of the game: 1 = first, 2 = second, 3 = third, 4 = fourth. |
| `catch_and_shoot` | 1 = the player catches a pass and shoots without first bouncing the ball (dribbling); 0 = otherwise. |
| `is_home` | 1 = the shooter's team is playing at its own venue; 0 = at the opponent's venue. |
| `zone` | Area of the court from which the shot is taken; labels are explained below. |
| `points` | Points actually scored by the shot: 0, 2, or 3. |
| `shot_id` | Unique identifier for the shot; not a predictor in this assignment. |
| `player` | Fictional name of the player taking the shot; not a predictor in this assignment. |

**Zone labels**

| Label | Meaning |
|---|---|
| `restricted` | The small area immediately around the basket. |
| `paint` | The rectangular area near the basket, excluding the restricted area. |
| `midrange` | Outside the paint but inside the three-point line. |
| `corner3` | A three-point shot from either corner of the court. |
| `abovebreak3` | A three-point shot from beyond the curved part of the three-point line, away from the corners. |

Shots in the first three zones are worth 2 points if made; shots in the last two are worth 3. Every miss earns 0 points.

**Result tables.** For all questions, use code to calculate results and generate requested tables; a displayed Pandas DataFrame is sufficient. No special formatting is needed.

## Q1. Define the task and prepare the training and test sets (2 points)

Read the CSV without sorting or dropping rows.

(a) **(0.5)** State the target and whether this is regression or classification. A teammate uses `points` as an input and reports nearly perfect validation accuracy. Is this appropriate for predicting a shot's outcome before it is taken? Explain. No model fitting is needed for this part.

(b) **(0.75)** Prepare two feature tables: *distance only* (`dist_ft`) and *full*, with these inputs in order: `dist_ft`, `defender_ft`, `shot_clock`, `period`, `catch_and_shoot`, `is_home`, `zone`.

Read the `pd.get_dummies` documentation. Use it to encode **only `zone`**, keeping one indicator column per category, in alphabetical category order after the six numeric columns. Briefly explain what the resulting columns and values mean. Do this **before splitting the data**. Let `X` be the full feature DataFrame and `y` the target Series, `made`. Keep the original `zone` column available for the summaries in Q2.

(c) **(0.75)** Randomly assign **75% of the shots to the training set and 25% to the test set**, using `train_test_split` with `random_state=332`. Keep other settings at their defaults. Keep the **same training and test set allocation for every model**. Report the numbers of training and test shots, plus the shape and column names of each training feature table.

Use only training data in Q2. Leave test outcomes unused until Q5.

## Q2. How do location and defensive pressure relate to success? (1.5 points)

(a) **(0.75)** For each `zone`, report `zone`, `n_shots` (number of shots), and `make_rate` (fraction made) in a table, sorted from highest to lowest make rate. Plot these rates as a bar chart in the same order, with labeled axes and a vertical scale of 0–1.

(b) **(0.75)** Combine all attempts from `corner3` and `abovebreak3`, including misses. Define `open_shot` as True when `defender_ft >= 6`, and False otherwise. Summarize the combined attempts in two rows: one for `open_shot=True` and one for `open_shot=False`. Report `open_shot`, `n_shots` (number of attempts in the group), and `make_rate` (fraction of the group's attempts made).

# Part B: Compare Models Without Using the Test Set (7.5 points)

Use **five-fold cross-validation (CV) on the training set from Q1**, with `cross_val_score`, `cv=5`, and accuracy as the scoring measure. Report the mean and standard deviation of the five validation accuracy scores as `cv_mean` and `cv_sd`.

Use the listed model settings; leave all others at their defaults.

## Q3. How much should we discourage large coefficients? (3.5 points)

Read the description of the `C` parameter in the LogisticRegression documentation. No derivation is needed.

(a) **(0.5)** Does a smaller `C` make regularization stronger or weaker? Predict whether `C=0.001` or `C=100` gives the smaller *sum of squared coefficients* with the full feature set. Explain your reasoning in 1–2 sentences.

(b) **(2)** Compare these five models, using `max_iter=2000` for logistic regression. For the majority baseline B, use `DummyClassifier(strategy="most_frequent")`, which predicts the most common training outcome.

| ID | Model | Features | C |
|---|---|---|---|
| B | Majority baseline | Full | Not applicable |
| LR1 | Logistic regression | Distance only | 1 |
| LR2 | Logistic regression | Full | 0.001 |
| LR3 | Logistic regression | Full | 1 |
| LR4 | Logistic regression | Full | 100 |

All logistic models use the **L2 penalty**. Do not scale columns or add features. Report `ID`, `features`, `C`, `cv_mean`, `cv_sd`, and `sum_sq_coef` in one table. To obtain `sum_sq_coef`, also fit each logistic model on the entire training set and sum its squared feature coefficients, **excluding the intercept**. Use a dash for B's `C` and `sum_sq_coef`.

Using your results, answer these three questions:

(i) How much does mean CV accuracy change when moving from distance-only LR1 to full-feature LR3? Report LR3 minus LR1, in percentage points.

(ii) Compare `sum_sq_coef` for LR2 (`C=0.001`) and LR4 (`C=100`). Which is smaller? Explain whether these values agree with your prediction in (a).

(iii) Among LR2–LR4, which model has the smallest `sum_sq_coef`, and which has the highest `cv_mean`? Are they the same model?

(c) **(1)** Use LR1 fitted on the entire training set. Report its intercept $a$ and distance coefficient $b$ to **6 decimals**. Use their unrounded values and the logistic-regression formula from class to calculate the make probability $p$ at 18 feet. Show your calculation in code and compare it with the probability for `made=1` from `predict_proba`. Report the predicted class at cutoffs 0.5 and 0.4: predict make when $p$ is at least the cutoff, and miss otherwise.

## Q4. Can a smaller tree make better predictions? (4 points)

Read about `max_depth` and `min_samples_leaf` in the DecisionTreeClassifier documentation. Use the full feature set and `random_state=332` for every tree. `max_depth=None` means no depth limit.

(a) **(0.5)** Explain what `min_samples_leaf=50` requires of each leaf. How do you expect increasing it from 1 to 50 to affect the number of leaves and training accuracy? Briefly explain your reasoning.

(b) **(1.5)** Compare these four trees:

| ID | max_depth | min_samples_leaf |
|---|---|---|
| T1 | 5 | 1 |
| T2 | 5 | 50 |
| T3 | None | 1 |
| T4 | None | 50 |

Evaluate each tree using CV. Separately, fit it on the entire training set to obtain its training accuracy and number of leaves. Look up how to obtain the leaf count. Report `ID`, `max_depth`, `min_samples_leaf`, `n_leaves`, `train_accuracy`, `cv_mean`, and `cv_sd` in one table. For each tree (T1–T4), plot two bars side by side: one for training accuracy and one for mean CV accuracy. Put the tree IDs on the horizontal axis and accuracy on the vertical axis, and include a legend.

(c) **(1)** Compare T1 with T2, then T3 with T4. For each pair, report both leaf counts and training accuracies, and explain whether these support your expectations in (a). Then compare their mean CV accuracies: did increasing `min_samples_leaf` improve validation performance? Does the tree with the highest training accuracy also have the highest mean CV accuracy among all four?

(d) **(0.75) How much depth helps?** Fit T5 with `max_depth=1` and `min_samples_leaf=50`. Add T5 to the table from (b), with the same columns; no additional chart is needed. Compare T5, T2, and T4: all use a minimum leaf size of 50, but allow depths of 1, 5, and no limit. How do training and mean CV accuracy change as the allowed depth increases? What do the results suggest about using a very shallow tree versus a very deep tree?

(e) **(0.25) Choose the final model.** From *all ten candidates* (B, LR1–LR4, T1–T5), choose the one with the highest **unrounded** mean CV accuracy. If candidates have exactly the same mean, choose the first of those candidates in this list. Report the selected ID, model, feature set, settings, and mean CV accuracy (6 decimals).

# Part C: Evaluate and Use Predictions (3.5 points)

Each question below specifies its own model or results; none depends on your Part B answers.

## Q5. Evaluate a specified model on the test set (2 points)

Use Q1's full feature set and training/test allocation. Fit a `DecisionTreeClassifier` with `max_depth=4`, `min_samples_leaf=50`, and `random_state=332` on the entire training set; leave other settings at their defaults. Evaluate this tree on the test set. Do not change the model or its settings after seeing the results.

(a) **(1)** Report test accuracy and a labeled confusion matrix: rows = actual, columns = predicted, both ordered miss (0), make (1).

(b) **(1)** Calculate precision and recall for `made=1`, showing your calculations. What does a false positive mean in this setting? What does a false negative mean?

## Q6. Compare two prediction rules (1.5 points)

Use only the supplied confusion matrix below, not your earlier results. It describes a separate model's predictions for test shots in `abovebreak3` with the nearest defender less than 6 feet away. Rows are actual outcomes; columns are predicted outcomes.

| Actual / predicted | Miss (0) | Make (1) |
|---|---|---|
| Miss (0) | 598 | 53 |
| Make (1) | 207 | 46 |

(a) **(1)** Report the total number of attempts and the number of actual makes. Compare the model with a rule that predicts *miss for every shot*. Report a two-row table with columns `method`, `accuracy`, and `recall`, treating `made=1` as the positive class. Show the numerical numerator and denominator used to calculate each of the four metric values.

(b) **(0.5)** A teammate argues that the method with higher accuracy is also better at identifying actual makes. Do you agree? Use the numbers of makes and misses and both methods' recall values to explain your answer.

## Contribution table (page 1 of your PDF)

Fill this in for your group and submit it as **page 1 of your PDF**, ahead of your notebook. List each member, what they worked on, and their approximate share of the total work. **The shares must add up to 100%.** Report the group's actual division of work; every member is expected to contribute roughly equally, and contact the head TA if contributions are substantially unequal.

| Member | What they worked on | Rough share |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | **Total** | **100%** |

*Use three rows if your group is three.*